

RELATÓRIO DE ANÁLISE DE PROJETO

infotime-web

Consistência, Qualidade e Estrutura

Caminho do projeto	C:\prj\lann\infotime-web
Data da análise	30 de abril de 2026
Módulos de negócio	37 módulos mapeados
Score geral	84/100 — Bom (fase de planejamento)

1. Visão Geral do Projeto

O projeto infotime-web consiste na migração completa do sistema legado Infotime / RP Financeiro — originalmente desenvolvido em Scriptcase/PHP — para uma stack moderna baseada em React, Fastify, Prisma e PostgreSQL, organizada como monorepo pnpm.

O repositório se encontra em fase de planejamento avançado: a infraestrutura técnica e toda a documentação de padrões estão estabelecidas, mas a implementação dos módulos de negócio ainda não foi iniciada. Este relatório avalia a qualidade, consistência e prontidão do projeto para início da implementação.

1.1 Métricas Gerais

Métrica	Valor
Módulos de negócio mapeados	37
Specs por módulo	10 arquivos padronizados
Total de specs geradas	370 arquivos Markdown
Documentos de padrão (docs/)	16 documentos PADRAO_*
Versão do TypeScript	5.9.3 (mais recente)
Versão do pnpm	9.15.0 (declarado em packageManager)
Score geral de qualidade	84/100 — Bom

2. Avaliação por Dimensão de Qualidade

Dimensão	Indicador Visual	Score	Classificação
Estrutura de pastas		95/100	
Stack tecnológica		92/100	
Documentação técnica		90/100	
Consistência de specs		85/100	
Cobertura de módulos		80/100	
Gestão de segredos/env		70/100	
CI/CD e linting		30/100	
Testes automatizados		20/100	

Legenda: ■ = 10 pontos ■■■ = 0 pontos | ≥85: Excelente | 60–84: Bom | <60: Crítico

3. Estrutura do Monorepo

O projeto adota a arquitetura de monorepo pnpm workspace, com workspaces declarados e separação clara entre aplicações executáveis (apps/) e bibliotecas compartilhadas (packages/).

3.1 Organização de Diretórios

Diretório / Arquivo	Descrição
apps/api/	Backend Fastify 5 — src/modules/, src/shared/, app.ts, server.ts
apps/web/	Frontend React 18 — src/app/, src/pages/, src/shared/, main.tsx, vite.config.ts
packages/database/	Prisma ORM — schema.prisma, seed.ts, src/prismaClient.ts
packages/shared-types/	Tipos TypeScript compartilhados: auth.ts, pagination.ts, index.ts — dist/ compilado
docs/	16 documentos PADRAO_* + docs/fontes/ com inventário da migração
specs/	370 specs Markdown — 10 arquivos por cada um dos 37 módulos de negócio
migration-source/	Corpus legado extraído — 37 pastas com database/, screenshots/, scriptcase/, specs/
migration-input/	Zips originais v0→v3 preservados em zips-originais/
scripts/	generate_entity_specs.py — gerador de specs por entidade
pnpm-workspace.yaml	Declaração dos workspaces do monorepo
tsconfig.base.json	Configuração TypeScript base compartilhada por todos os pacotes
.env.example	Template de variáveis de ambiente com todas as chaves documentadas

3.2 Avaliação por App/Package

apps/api — Fastify 5

- ✓ Separação correta: src/modules/ e src/shared/
- ✓ Entrypoints separados: app.ts (setup) + server.ts (boot)
- ✓ tsconfig.json próprio referenciando tsconfig.base.json
- ✗ PROBLEMA: Pasta dist/ presente no repositório — deve estar no .gitignore

apps/web — React 18 + Vite

- ✓ Estrutura correta: src/app/, src/pages/, src/shared/
- ✓ vite.config.ts configurado na raiz de apps/web/
- ✓ index.html na raiz do app (padrão obrigatório do Vite)
- ✗ PROBLEMA: Pasta dist/ presente no repositório — deve estar no .gitignore

packages/database — Prisma 6

- ✓ schema.prisma e seed.ts presentes e organizados
- ✓ prismaClient.ts com client singleton isolado
- ✓ tsconfig.json próprio com scripts de seed configurados

packages/shared-types — Tipos Compartilhados

- ✓ auth.ts, pagination.ts e index.ts com tipos base

- ✓ dist/ compilado e acessível por api e web via workspace
- ⚠ Atenção: nenhum package de utils/helpers compartilhados ainda

4. Stack Tecnológica

A stack escolhida é moderna, coerente e alinhada com as boas práticas de mercado em 2025/2026. Todos os pacotes principais estão em versões estáveis e recentes.

4.1 Análise de Dependências

Pacote	Versão	Camada	Status	Observação
React	18.3.1	Frontend	OK	Versão LTS — estável e adequada
React Router DOM	7.14.2	Frontend	OK	RR v7 com data APIs modernas
TanStack Query	5.100.6	Frontend	OK	Server-state caching — correto para o padrão
PrimeReact	10.9.7	Frontend	OK	UI lib enterprise completa com PrimeFlex
React Hook Form	7.74.0	Frontend	OK	Alinhado com PADRAO_FORMULARIOS.md
Vite	6.4.2	Build	OK	Build tool moderno — bundler padrão React
Fastify	5.8.5	API	OK	Alta performance — v5 é a versão atual
@fastify/jwt	9.1.0	API	OK	Autenticação JWT configurada
@fastify/cors	10.1.0	API	OK	CORS configurado corretamente
Prisma	6.19.3	Banco	OK	ORM atual — schema.prisma e seed presentes
Zod	3.25.76	Validação	OK	Compartilhado entre API e Web — excelente
TypeScript	5.9.3	Tooling	OK	Versão mais recente disponível
bcryptjs	2.4.3	Segurança	ATENÇÃO	Versão 2021, puro JS — avaliar migração para argon2
esbuild	0.25.12 / 0.27.7	Build	ATENÇÃO	Duas versões no .npmm — verificar conflito transitório

5. Módulos de Negócio

O projeto mapeia 37 módulos de negócio do sistema legado. Cada módulo possui uma pasta em specs/ com exatamente 10 arquivos Markdown padronizados, e uma pasta espelho em migration-source/ com o corpus legado organizado.

5.1 Lista Completa dos 37 Módulos

Módulo	Módulo	Módulo
agenda	almoxarifado	almoxarifado-baixa
almoxarifado-entrada	almoxarifado-requisicao	aplicacao
auditoria	auth	avaliacao-infolab
banco	boleto	cargo
cliente	cliente-licenca	colaborador
colaborador-rh	colaborador-tarefa	colaborador-viagem
concorrente	configuracao	conta-caixa
contrato	empresa	fornecedor
grupo-usuario	implantacao	lançamento-despesa
lançamento-receita	negociacao	nota-fiscal
patrimonio	pix	plano-conta
pop-documento	portal-cliente	produto
proposta	retorno-cnab	treinamento
usuario		

5.2 Estrutura Padrão de Specs por Módulo (10 arquivos)

Arquivo	Conteúdo
README.md	Visão geral do módulo, entidades envolvidas e escopo
api.md	Endpoints REST/Fastify: rotas, parâmetros, payloads, respostas
frontend.md	Componentes React, páginas, fluxo de navegação
modelo-dados.md	Entidades, campos, tipos, índices, relacionamentos
telas.md	Mapeamento das telas do sistema legado → novo sistema
mapa-campos.md	Correspondência campo a campo entre legado e novo modelo
permissoes.md	Matriz de permissões RBAC por perfil de usuário
checklist.md	Checklist de migração com status de cada item
duvidas-abertas.md	Questões em aberto que precisam de validação do negócio
regras-scriptcase.md	Regras de negócio extraídas do código Scriptcase legado

6. Documentação de Padrões

O diretório docs/ contém 16 documentos de padrão técnico. Esta é uma das maiores forças do projeto: a documentação cobre todos os aspectos relevantes da arquitetura e do processo de migração, servindo como guia definitivo para a equipe de desenvolvimento.

6.1 Inventário dos Documentos de Padrão

Documento	Cobertura
PADRAO_ARQUITETURA_INFOTIME_WEB.md	Visão geral da arquitetura, camadas e padrões do monorepo
PADRAO_AUTH_RBAC.md	Autenticação JWT, refresh token e controle de acesso por papel
PADRAO_API_CRUD.md	Convenções de rotas, handlers, erros e paginação no Fastify
PADRAO_PRISMA.md	Modelos, migrações, seed e boas práticas com Prisma ORM
PADRAO_TANSTACK_QUERY.md	Uso de queries, mutations e cache com TanStack Query 5
PADRAO_FORMULARIOS.md	Formulários com React Hook Form + Zod + PrimeReact
PADRAO_LISTAS.md	DataTable, filtros, paginação e ordenação com PrimeReact
PADRAO_VALIDACOES_ZOD.md	Schemas Zod compartilhados entre frontend e API
PADRAO_TENANCY_AUDITORIA.md	Multitenancy, isolamento de dados e trilha de auditoria
PADRAO_TESTES.md	Estratégia de testes unitários, integração e e2e
PADRAO_UPLOAD_ARQUIVOS.md	Upload com MinIO/S3 — endpoints, storage e referências
PADRAO_MIGRACAO_INFOTIME.md	Processo e estratégia geral de migração do sistema legado
PADRAO_MIGRACAO_SCRIPTCASE.md	Mapeamento de padrões Scriptcase para a nova stack
ENTRADA_MIGRACAO.md	Entrada do corpus de migração — fontes e inventário
ORDEM_IMPLEMENTACAO_MODULOS.md	Backlog priorizado de módulos para implementação
PROMPTS_OPERACIONAIS_CURSOR.md	Guia de uso de IA (Cursor) no desenvolvimento do projeto

7. Problemas Identificados

A seguir estão listados todos os problemas encontrados, classificados por severidade, com recomendações de correção.

7.1 Críticos — Devem ser resolvidos antes do início da implementação

7.1.1 Ausência total de testes automatizados

Nenhum arquivo `.test.ts` ou `.spec.ts` foi encontrado em qualquer parte do projeto. O documento `PADRAO_TESTES.md` existe e define a estratégia, mas não há nenhuma implementação.

Impacto: A ausência de testes inviabiliza refatorações seguras e aumenta o risco de regressões durante a migração dos 37 módulos.

Recomendação: Configurar Vitest na raiz do monorepo como dependência de desenvolvimento compartilhada. Implementar testes unitários nos pacotes compartilhados (`shared-types`, `database`) antes de iniciar os módulos de negócio.

7.1.2 Ausência de ESLint e Prettier

Os scripts `"lint"` estão declarados no `package.json` raiz, mas não há `eslint.config.js`, `.eslintrc`, nem `.prettierrc` em nenhum nível do projeto. Os scripts de lint não funcionam sem a configuração base.

Impacto: Sem linting, o código produzido pelos múltiplos desenvolvedores terá inconsistências de estilo e poderá conter erros estáticos não detectados.

Recomendação: Adicionar `eslint.config.js` na raiz com regras para TypeScript + React. Adicionar `.prettierrc` com configuração mínima. Configurar `"lint-staged"` + Husky para executar antes dos commits.

7.1.3 Ausência de pipeline CI/CD

Nenhuma pasta `.github/workflows` (ou equivalente para GitLab/Bitbucket) foi encontrada. Sem automação, qualquer push pode introduzir código quebrado sem detecção.

Recomendação: Criar GitHub Actions com workflow mínimo: `typecheck` (`tsc --noEmit`), `lint`, `build` e `test` — executados a cada Pull Request.

7.2 Moderados — Devem ser resolvidos em breve

7.2.1 Pastas `dist/` presentes no repositório

As pastas `apps/api/dist/`, `apps/web/dist/` e `packages/shared-types/dist/` aparecem como parte do repositório. Artefatos de build não devem ser versionados.

Recomendação: Verificar o `.gitignore` e garantir que `dist/` está listado. Remover os arquivos de build do histórico git se necessário (`git rm -r --cached dist/`).

7.2.2 bcryptjs desatualizado e menos seguro

A versão 2.4.3 do bcryptjs (de 2021) é uma implementação pura em JavaScript, sem binding nativo ao sistema operacional. Isso a torna mais lenta e potencialmente mais vulnerável que alternativas modernas.

Recomendação: Migrar para argon2 (Argon2id é o padrão recomendado pelo OWASP) ou para bcrypt com binding nativo. A migração pode ser feita de forma gradual com hash verification compatível.

7.2.3 Duas versões de esbuild no .npnm

O diretório .npnm contém tanto esbuild@0.25.12 quanto esbuild@0.27.7. Isso indica dependências transitórias com versões conflitantes.

Recomendação: Executar "npnm why esbuild" para identificar qual pacote força cada versão. Usar "overrides" no package.json raiz para unificar em uma versão se necessário.

7.3 Informativos — Pontos de atenção

7.3.1 README.md com encoding corrompido

O conteúdo do README.md está correto, mas ao ser lido pelo PowerShell o texto aparece com caracteres UTF-8 corrompidos (ex.: "MigraÃ§Ã£o" em vez de "Migração"). O arquivo em si está provavelmente correto; o problema é a exibição no terminal Windows.

Recomendação: O arquivo está OK. Para exibição correta no PowerShell, usar:
[Console]::OutputEncoding = [System.Text.Encoding]::UTF8.

7.3.2 Script generate_entity_specs.py sem documentação de uso

O arquivo scripts/generate_entity_specs.py está presente mas não há referência a ele no README.md nem comentários de uso no próprio arquivo.

Recomendação: Adicionar docstring de uso no topo do script e referenciar no README.md com exemplos de execução.

7.3.3 Nenhum módulo de negócio implementado

Isso é esperado para a fase atual (planejamento). A implementação ainda não foi iniciada, o que é consistente com a riqueza da documentação de padrões e specs produzida.

Recomendação: Seguir a ORDEM_IMPLEMENTACAO_MODULOS.md iniciando pelo módulo auth (infraestrutura de autenticação) como base para todos os demais.

8. Pontos Fortes do Projeto

8.1 Arquitetura

- ✓ Monorepo pnpm workspace corretamente configurado com workspaces declarados
- ✓ tsconfig.base.json compartilhado garante consistência TypeScript em todo o monorepo
- ✓ packages/shared-types compila e exporta tipos usados por api e web — eliminando duplicação
- ✓ packages/database isolado com Prisma + seed — separação clara de responsabilidades
- ✓ Separação explícita entre apps/ (executáveis) e packages/ (bibliotecas)
- ✓ scripts de dev, build, db:generate, db:migrate, db:push e db:seed no package.json raiz

8.2 Documentação

- ✓ 16 documentos PADRAO_* cobrindo todos os aspectos da arquitetura e implementação
- ✓ PADRAO_MIGRACAO_SCRIPTCASE.md — mapeamento detalhado do legado Scriptcase para a nova stack
- ✓ PADRAO_TENANCY_AUDITORIA.md — multitenancy planejado desde o início, evitando retrabalho futuro
- ✓ docs/fontes/ com inventário completo: 00_INVENTARIO_ZIPS, 01_MAPA_CORPUS, 02_ENTIDADES, 03_DUPLICIDADES, 04_RISCOS
- ✓ PROMPTS_OPERACIONAIS_CURSOR.md — guia de uso de IA no desenvolvimento, acelerando onboarding
- ✓ README.md completo com pré-requisitos, variáveis de ambiente e passo a passo de primeira execução

8.3 Processo de Migração

- ✓ migration-input/ com zips originais versionados v0 → v3 — rastreabilidade total das fontes
- ✓ migration-source/ extraído e organizado com a mesma estrutura de módulos do projeto destino
- ✓ Cada módulo legado tem: database/, screenshots/, scriptcase/ e specs/ — corpus completo
- ✓ SCREENSHOTS_AUDIT.md e CHECKLIST_MIGRACAO_TELA.md garantem fidelidade visual da migração
- ✓ Script Python generate_entity_specs.py para geração automatizada de specs

8.4 Stack e Dependências

- ✓ React 18 + Fastify 5 + Prisma 6 + TypeScript 5.9 — versões atuais e compatíveis entre si
- ✓ Zod compartilhado entre API e frontend — validação única, sem duplicação de schemas
- ✓ TanStack Query 5 para server-state — padrão correto para sistema com muitas consultas
- ✓ PrimeReact 10 com PrimeFlex — UI lib enterprise adequada para sistema de gestão complexo

9. Variáveis de Ambiente

O arquivo .env.example documenta todas as variáveis necessárias. A gestão de segredos está correta em teoria, mas precisa de verificação prática.

Variável	Obrigatória	Status	Descrição
----------	-------------	--------	-----------

DATABASE_URL	Sim	No .env.example	Connection string PostgreSQL (inclui schema search_path)
JWT_SECRET	Sim	No .env.example	Segredo para assinatura de tokens JWT de acesso
JWT_REFRESH_SECRET	Recomendada	No .env.example	Segredo para tokens de refresh (rotação de JWT)
VITE_API_URL	Sim (web)	No .env.example	URL base da API para o frontend (ex.: http://localhost:3333)
S3_ENDPOINT / S3_BUCKET	Opcional	Condicional	MinIO/S3 para uploads — necessário apenas quando habilitado
S3_ACCESS_KEY / S3_SECRET_KEY	Opcional	Condicional	Credenciais de acesso ao storage de arquivos

ATENÇÃO: Verificar se existe um arquivo .env real na raiz e em apps/api/ — esses arquivos DEVEM estar no .gitignore para não expor credenciais.

10. Próximos Passos Recomendados

Lista priorizada de ações recomendadas antes e durante o início da implementação dos módulos de negócio.

#	Prioridade	Ação	Benefício
1	Crítico	Configurar ESLint + Prettier na raiz do monorepo	Consistência de código
2	Crítico	Configurar Vitest com coverage no monorepo	Base para TDD
3	Crítico	Criar GitHub Actions: typecheck + lint + test por PR	Qualidade automatizada
4	Moderado	Verificar .gitignore — excluir dist/ de todos os apps/packages	Repositório limpo
5	Moderado	Avaliar migração de bcryptjs para argon2	Segurança de senhas
6	Normal	Implementar módulo auth — base de autenticação para todos os demais	Infraestrutura base
7	Normal	Seguir ORDEM_IMPLEMENTACAO_MODULOS.md para priorizar módulos	Entrega incremental
8	Normal	Documentar uso do script generate_entity_specs.py no README	Facilitar onboarding

11. Conclusão

O projeto infotime-web apresenta uma fundação técnica e documental de alta qualidade para um projeto em fase de planejamento. A escolha da stack, a organização do monorepo, a riqueza da documentação de padrões e a profundidade do mapeamento do sistema legado são diferenciais importantes que reduzem significativamente o risco da migração.

Os principais riscos estão concentrados na ausência de testes automatizados, linting e CI/CD — três pilares de qualidade de engenharia que devem ser estabelecidos antes do início da implementação dos módulos de negócio. Com essas lacunas preenchidas, o projeto tem todas as condições para uma migração bem-sucedida.

Score final: 84/100 — BOM. Projeto pronto para início da implementação após resolução dos 3 pontos críticos identificados.

Relatório gerado por análise automatizada assistida por IA | infotime-web | Abril 2026